2026-7-12 Linux进程--理解

- 1. 从程序文件理解进程
  - 1.1 磁盘上的程序不是进程
    - 1.1.1 程序和可执行文件仍然是静态对象
    - 1.1.2 启动命令或应用是在创建进程
    - 1.1.3 同一个可执行文件可以产生多个进程
  - 1.2 进程由“程序内容”和“管理记录”共同构成
    - 1.2.1 仅有代码和数据还不够
    - 1.2.2 用两个视角理解进程
- 2. 从 task_struct 理解进程
  - 2.1 PCB 是操作系统眼中的进程档案
    - 2.1.1 PCB 记录进程是什么、处于什么状态
    - 2.1.2 Linux 用 task_struct 实现 PCB
    - 2.1.3 “简历”类比只用于帮助理解
  - 2.2 从属性看内核为什么需要 task_struct
    - 2.2.1 标识与状态让进程可区分
    - 2.2.2 执行信息让进程可以暂停后继续
    - 2.2.3 资源信息让操作系统能够统一管理
- 3. 从 CPU 执行理解进程
  - 3.1 程序不是一口气执行完的
    - 3.1.1 程序计数器决定下一条指令
    - 3.1.2 时间片结束时不一定要执行完
  - 3.2 进程切换依赖上下文保存与恢复
    - 3.2.1 调度决定下一个进程，切换负责换现场
    - 3.2.2 CPU 寄存器只有当前现场，进程上下文可以有多份
    - 3.2.3 切换的本质是让执行流连续地接上
  - 3.3 进程怎样取得自己的身份信息
    - 3.3.1 内核属性不能由用户进程直接读取
    - 3.3.2 getpid 与 getppid
- 4. 如何整体理解进程
  - 4.1 一个进程同时具有静态内容和动态状态
  - 4.2 核心理解链路
  - 4.3 自检问题

本篇范围

> 前置阅读：2026-7-7 Linux进程--基础概念。本篇只回答“怎样从运行过程理解进程”：程序如何变成进程、`task_struct` 为什么是进程档案、CPU 为什么不能把一个进程一口气执行完，以及进程如何取得自己的身份信息。进程状态、调度算法和完整切换流程留到后续专题。

1. 从程序文件理解进程

1.1 磁盘上的程序不是进程

1.1.1 程序和可执行文件仍然是静态对象

源代码和编译后的可执行文件都可以放在磁盘上。例如：

```
type -a ls
ls -l /usr/bin/ls
```

`ls` 最终对应磁盘上的一个可执行文件。它可以被查看、复制、删除，也可以在获得执行权限后被启动，但仅仅“存在于磁盘上”时，它还没有获得 CPU 执行权，也没有自己的运行状态。

因此要区分三种对象：

对象	所处位置	主要特点
源程序	磁盘	人编写的代码和数据
可执行文件	磁盘	已经编译、链接，可以启动
进程	内存和操作系统管理结构中	正在执行，并拥有独立的运行状态

可执行文件不是进程；它是创建进程时使用的程序内容。

1.1.2 启动命令或应用是在创建进程

双击一个程序、执行一条命令，或在终端启动一个应用，本质都是请求操作系统创建一个新的运行实体。

从文件角度看，启动过程可以概括为：

```
磁盘上的可执行文件
        ↓ 加载
内存中的代码和数据
        ↓ 内核建立管理信息
一个可以被 CPU 执行的进程
```

操作系统需要把程序的代码和数据装入适当的内存区域，同时建立描述该进程的内核数据结构。从此以后，操作系统不再只是“看见一个文件”，而是管理一个具有身份、执行位置和资源状态的进程。

1.1.3 同一个可执行文件可以产生多个进程

同一个可执行文件可以被启动多次，每次启动通常形成一个独立进程。它们可能执行相同的代码，却拥有各自的：

- 进程标识符；
- 运行状态；
- 当前执行位置；
- 寄存器和栈中的运行数据；
- 打开的文件和其他资源。

所以“进程就是程序”的说法不够准确。更准确的理解是：

> 可执行文件提供程序内容，进程则是这个程序在一次具体执行中的运行实例。

1.2 进程由“程序内容”和“管理记录”共同构成

1.2.1 仅有代码和数据还不够

假设操作系统只把代码和数据放进内存，却不记录任何其他信息，那么系统将无法回答：

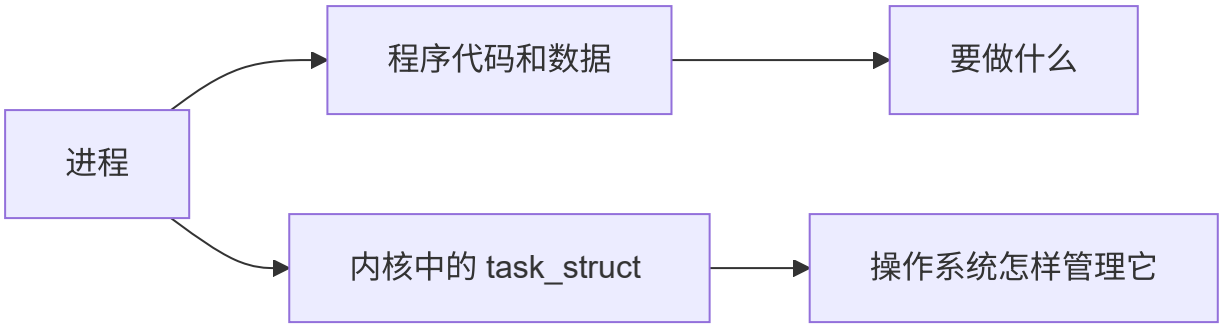
- 这个进程的 PID 是多少？
- 它上一次执行到了哪一条指令？
- 它现在能不能继续运行？
- CPU 下一次应该选择哪个进程？

- 它占用了哪些内存和文件？

代码和数据描述“要执行什么”，内核管理记录描述“这次执行现在处于什么状态”。二者共同组成完整的进程。

1.2.2 用两个视角理解进程

可以把进程拆成两个互补的部分：



因此：

```
进程 = Linux 内核中的 task_struct + 该进程自己的代码和数据
```

`task_struct` 不替代代码和数据，代码和数据也不替代 `task_struct`。前者让操作系统“知道并管理这个进程”，后者让进程“有实际可执行的内容”。

2. 从 task_struct 理解进程

2.1 PCB 是操作系统眼中的进程档案

2.1.1 PCB 记录进程是什么、处于什么状态

操作系统管理进程时，采用的基本方法仍然是“先描述，再组织”。

- **描述**：为每个进程建立 PCB，记录它有哪些属性；
- **组织**：把多个 PCB 放入链表、队列或其他数据结构中统一管理。

可以把 PCB 理解成进程的档案。档案中不仅有名字，还记录状态、能力和历史信息。操作系统通过查看档案来管理进程，而不需要把进程的一切都塞进 CPU。

2.1.2 Linux 用 task_struct 实现 PCB

PCB 是通用概念，Linux 中的具体实现是 `task_struct`。

```
PCB：抽象的进程控制块概念
task_struct：Linux 内核对 PCB 的具体数据结构实现
```

每个被管理的进程都对应一份内核中的 `task_struct`。它包含进程标识、状态、优先级、程序计数器、内存指针、上下文数据、I/O 信息和记账信息等内容。

2.1.3 “简历”类比只用于帮助理解

课堂上常把 `task_struct` 类比成简历：简历集中记录一个人的身份、经历和能力，方便别人快速了解并作出判断；`task_struct` 则集中记录一个进程的身份和运行属性，方便内核识别、调度和分配资源。

这个类比只说明“集中描述”的思想。真正执行程序的是进程本身，简历或 PCB 都不能代替实际的代码和数据。

2.2 从属性看内核为什么需要 task_struct

2.2.1 标识与状态让进程可区分

`task_struct` 保存 PID 等标识信息，使不同进程可以被区分；保存运行状态、退出代码和信号等信息，使内核知道当前应该怎样处理它。

如果没有这些信息，系统虽然拥有多段代码和数据，却无法可靠地判断哪一段属于谁，也无法决定下一步做什么。

2.2.2 执行信息让进程可以暂停后继续

进程在 CPU 上执行时，会把指令地址、临时计算结果等重要信息放在寄存器和栈中。进程暂时离开 CPU 后，这些现场不能丢失。

因此，进程管理结构需要保存或关联：

- 程序计数器：下一条应该执行的指令在哪里；

- 寄存器上下文：当前执行现场是什么；
- 栈等信息：函数调用和临时数据怎样恢复。

这些信息共同回答一个问题：

> 这个进程已经被执行到哪里了，下一次被选中时应该从哪里继续？

2.2.3 资源信息让操作系统能够统一管理

内存指针、文件和 I/O 状态等信息，使内核能够知道进程正在使用哪些资源。此后无论进行资源统计、等待 I/O，还是切换进程，都可以围绕同一份进程管理记录展开。

这也是“进程是资源分配对象”的具体体现：进程不仅是一段代码，还带着一组属于本次执行的资源关系。

3. 从 CPU 执行理解进程

3.1 程序不是一口气执行完的

3.1.1 程序计数器决定下一条指令

CPU 执行代码时会不断取下一条指令。程序计数器（Program Counter，PC）记录下一条即将执行指令的位置。

函数调用能说明这一点。下面的程序中，`main()` 调用 `add()`：

```c
#include <stdio.h>

int add(int x, int y)
{
    int z = x + y;
    return z;
}

int main(void)
{
    int result = add(2, 3);
    printf("result: %d\n", result);
```

```
    return 0;
}
```

执行到调用 `add()` 时，CPU 需要转到函数入口执行；函数返回后，又要回到调用点之后继续执行。程序计数器、寄存器和栈共同保证这条执行路径不会丢失。

函数调用仍然属于同一个进程，并不会因为调用了 `add()` 就产生新进程。

3.1.2 时间片结束时不一定要执行完

一个进程获得 CPU 后，并不需要一直执行到程序结束才离开 CPU。现代分时操作系统通常会为进程分配一个**时间片**：

```
运行至时间片结束 -> 保存当前现场 -> CPU 执行其他进程
```

时间片用完时，当前进程可能还有很多代码没有执行。操作系统会暂时把它从 CPU 上取下，让其他进程获得运行机会。

这就是“基于时间片的轮转”的基本含义：多个进程轮流使用 CPU，在宏观上表现为“同时推进”，但它们并不是在同一个 CPU 核心上真正同时执行。

3.2 进程切换依赖上下文保存与恢复

3.2.1 调度决定下一个进程，切换负责换现场

可以先区分两个相关动作：

- **调度**：操作系统根据策略决定下一次让哪个进程运行；
- **切换**：把当前进程的现场保存起来，再恢复下一个进程的现场。

只有先选出目标进程，系统才知道应该恢复哪一份上下文；而恢复之后，目标进程才能从上次离开的位置继续执行。

3.2.2 CPU 寄存器只有当前现场，进程上下文可以有多份

同一时刻，一个 CPU 核心中的寄存器只能保存一套当前值。但多个进程各自都可能拥有自己的上下文副本。

进程离开 CPU 时：

```
当前寄存器现场 -> 保存为该进程的上下文
```

新进程进入 CPU 时：

```
新进程的上下文 -> 恢复进寄存器
```

因此可以这样理解：

> CPU 寄存器只有一套“当前现场”，但每个进程都可以保存一份属于自己的“历史现场”。上下文属于进程私有，不和其他进程混用。

`task_struct` 记录或关联这些上下文信息；临时运行数据还会由内核栈等结构承接。具体保存了哪些寄存器、切换时内核执行了哪些步骤，后续再结合进程切换专题展开。

3.2.3 切换的本质是让执行流连续地接上

进程切换不会重新从头执行程序。它会把离开前的 PC、寄存器和其他必要现场恢复回来，使进程从中断处继续。

所以从进程自己的角度看，它像是“一直向前运行”；从 CPU 的角度看，它可能已经暂停过很多次，期间还执行过其他进程。

> **本篇的深度边界**
>
> 这里只需要理解“时间片、保存上下文、恢复上下文”这条主线。完整的调度队列、优先级算法和切换代码属于后续“进程切换和调度”专题。

3.3 进程怎样取得自己的身份信息

3.3.1 内核属性不能由用户进程直接读取

PID、PPID 等信息保存在内核管理的 `task_struct` 中。普通进程不能任意读取内核内部的 `task_struct`，因此操作系统需要提供受控接口。

这个接口就是系统调用。进程调用系统调用进入内核，由内核读取对应进程的属性，再把需要的结果返回给用户态程序。

3.3.2 getpid 与 getppid

Linux 提供：

- `getpid()`：取得当前进程的 PID；
- `getppid()`：取得当前进程的父进程 PID。

可以先查看系统调用说明：

```
man 2 getpid
```

最小示例：

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main(void)
{
    printf("pid  = %d\n", getpid());
    printf("ppid = %d\n", getppid());
    return 0;
}
```

编译并运行后，可以再通过进程查看工具核对结果：

```
gcc getpid_demo.c -o getpid_demo
./getpid_demo
ps -p PID -o pid,ppid,stat,comm
```

示例中的 `PID` 要替换成程序实际打印出的数字。这里要理解的重点不是记住某次输出，而是这条链路：

```
用户进程调用 getpid()
        ↓
```

```
内核读取该进程的管理信息
        ↓
将 PID 返回给用户态
```

4. 如何整体理解进程

4.1 一个进程同时具有静态内容和动态状态

可以从两个层面看进程：

层面	要回答的问题
程序内容	这个进程要执行哪些代码、处理哪些数据?
内核管理	这个进程是谁、现在怎样、应该从哪里继续、占有哪些资源?

这两个层面合起来，才是完整意义上的进程。

4.2 核心理解链路

```
磁盘上的可执行文件
        ↓ 启动
代码和数据被装入内存
        ↓ 操作系统描述并组织
内核创建 task_struct
        ↓ 调度
进程获得一段时间片，在 CPU 上执行
        ↓ 时间片结束
保存上下文，切换其他进程
        ↓ 再次被调度
恢复上下文，从程序计数器指示的位置继续
```

可以用一句话收束：

> 进程是“程序内容 + 内核管理记录 + 当前执行状态”组成的运行实体；操作系统通过 `task_struct` 管理它，CPU 则通过上下文切换让多个执行流交替推进。

4.3 自检问题

应该能够用自己的话回答

1. 为什么磁盘上的可执行文件还不是进程?
2. 为什么 `task_struct` 不能单独组成一个完整进程?
3. 程序计数器和寄存器上下文分别帮助恢复什么?
4. 时间片结束时，进程为什么没有执行完也会离开 CPU?
5. 为什么 CPU 寄存器只有一套，却可以保存多个进程的上下文?
6. `getpid()` 为什么要通过系统调用取得信息?